

A dynamic TDMA based scheme for securing query processing in WSN

Amrita Ghosal · Subir Halder · Sipra DasBit

Published online: 29 October 2011
© Springer Science+Business Media, LLC 2011

Abstract Nodes in a wireless sensor network (WSN) are generally deployed in unattended environments making the nodes susceptible to attacks. Therefore, the need of defending such attacks becomes a big challenge. We propose a scheme to build a security mechanism in a query-processing paradigm within WSN. The scheme is capable of protecting replay attack while preserving essential properties of security such as authentication, data integrity and data freshness. The solution is made lightweight using symmetric key cryptography with very short-length key. Further, the key used in our scheme is neither pre-deployed nor is transmitted directly. The key information is established among nodes through an efficient use of one variant of dynamic TDMA mechanism which ensures security of key. Another variant of dynamic TDMA is used to make the scheme bandwidth saving, an essential quality of WSN. Performance of the scheme is analyzed in terms of storage, computation and communication overhead. Finally the analytical results are compared with two of the existing schemes including the previous version of the present scheme that show significant reduction of all such overheads thereby proving the suitability of the proposed scheme for a resource-constrained network like WSN.

Keywords Dynamic TDMA · Replay attack · Denial of Service (DoS) attack · Authentication · Data integrity · Data freshness

1 Introduction

The advent of efficient short range radio communication coupled with advances in miniaturization of computing devices has grown profound interest in wireless sensor networks [1, 2]. In most of the applications, nodes in a wireless sensor network (WSN) are deployed in an open area, thereby remain unattended and are prone to attack [3–5] by adversaries. Therefore, the need of ensuring security while communicating in presence of such attacks is of utmost importance. However, traditional security measures designed for the resource-rich networks such as LAN etc. are not suitable for the WSN, a resource-constrained network. These constraints arise out of limitation in computational power of the nodes, limitation in communication over the unguided medium and limitation in deployment strategies.

In presence of such limitations it becomes mandatory to devise lightweight security solutions for wireless sensor networks (WSNs). Public key cryptography is not preferred as it leads to complex computations creating a barrier to make the solution lightweight. Therefore, symmetric key cryptography is employed as a security solution that requires relatively less computations and memory requirement is also minimized.

Cluster based architecture [6], which is one of the most commonly used energy-saving WSN architectures, is considered here. Ensuring secure functioning of such cluster based WSNs is essential to provide security both at the cluster heads (CHs) and at the member nodes (MNs). Many

A. Ghosal (✉) · S. Halder
Department of Computer Science & Engineering, Dr. B. C. Roy
Engineering College, Durgapur, India
e-mail: ghosal_amrita@yahoo.com

S. Halder
e-mail: subir_ece@rediffmail.com

S. DasBit
Department of Computer Science & Technology, Bengal
Engineering and Science University, Shibpur, India
e-mail: sdasbit@yahoo.co.in

works reported so far provide solutions for securing communication between cluster head and member nodes and/or between cluster head and base station (BS).

In one such work [7] which is based on LEACH protocol [6], all nodes are provided with two keys at the time of their deployment. The nodes selected as cluster heads are authenticated by the base station, which is trusted and has more resources. The authentication is done with the help of one key which is shared between BS and the node. Once the CHs are authenticated by the BS, it broadcasts the list of authenticated CHs. With the help of the other key, member nodes first authenticate the base station as the broadcaster and then accept the list to see the authenticated CHs to take a decision of joining as a member node (MN) of a cluster. The major advantage of the scheme is its capability to prevent an intruder which poses as a cluster head (CH). But the intruder can easily pose itself as an MN and set up DoS attacks [8] on CH by generating bogus packets and sending the same to it. In other words, no solution has been provided towards securing the communication between cluster head and member nodes.

Authors in both [9] and [10] have proposed schemes for selection of secure cluster heads. Both the schemes assume that the BS is trusted and has more resources. In [9], each node has three keys such as master key, cluster key and node-to-node keys. The master key facilitates broadcast by BS whereas the cluster key facilitates multicasting between BS and cluster members as well as communication within the clusters themselves. The node-to-node key helps intra nodes communication. In this scheme every node is evaluated by its neighbouring nodes that send an assessment in terms of information called trust value to the BS. The BS in turn selects the node with the highest trust value as the CH. The merit of this scheme is that it tries to develop a trust-based framework for clustered WSNs by introducing a mechanism that minimizes the chances of malicious nodes being elected as cluster heads. However, maintenance of huge number of keys and evaluation of these trust values require massive overhead for resource-constrained sensor nodes. In [10], all the nodes send AREP (Authentication Reply) messages to base station after receiving AREQ (Authentication Request) messages from the base station for a constant period of time. The BS selects CHs based on a parameter validity authentication rate (VAR) which is computed at BS with the help of round-wise transmitted authentication message of each node from the nodes to BS along with their residual power. The parameter VAR measures relative integrity and confidentiality of the nodes. The nodes, instead of sending authentication message for every round, periodically skip some consecutive rounds. During the skipped rounds BS continues CH selection process by using average of VAR computed from VARs of previous rounds thereby saving energy. The greatest

strength of the scheme is selecting relatively secured nodes as CHs as the need is more serious to protect CHs from attack than the other nodes. The demerits of both the schemes [8, 9] lie on the fact that the schemes are silent about securing communication between member node and cluster head.

In SecLEACH [11], authors have used random key predistribution method for assigning keys to all nodes aiming to prevent illegitimate nodes from participating in the network. The random key predistribution scheme has been used to secure communication between member node and cluster head. It implements the security scheme modifying the LEACH clustering protocol. All nodes are assigned a set of keys with the help of which each node discovers secure links with their neighbouring nodes. In setup phase, in response of the broadcast messages by the self-elected cluster heads, ordinary nodes (MNs) choose the closest CH with whom they share a key. Accordingly an MN sends a joining message protected by a message authentication code (MAC) generated at the MN in association with the shared key. The CH in turn sends the necessary time slots to the willing MNs. The MN to CH communication in steady state phase is also protected by the respective MAC and shared key. The main contribution of this work is securing dynamic cluster-based sensor network by employing random key pre-distribution which is widely used in flat network. However, at the end of clustering some of the nodes called as orphans will remain unattached with any of the CHs due to key sharing constraints. Although the scheme takes care of the orphans keeping them in sleep mode for the round, it has a serious impact on the performance of the network.

The scheme presented in [12] deals with authentication issues including detection and elimination of compromised nodes from WSN. The entire task of providing security solution is divided among three different modules. One module is designed for providing primary security, another is for intrusion detection and the other is for securing cluster formation. The cluster formation module takes care of CH selection and joining of ordinary nodes to a suitable CH to become an MN of the corresponding cluster. In the module to provide primary security, securing communication between MNs and CHs are implemented using their shared pair-wise keys. In intrusion detection module, whenever a node becomes suspicious about another node, an alarm packet is generated by the first node and is sent to the BS. The nodes may be an MN or a CH. The procedures followed in the cluster formation and primary security modules are quite complex. The complexity arises out of a number of packets/messages exchange such as ‘application’ packet, ‘joining’ packet, ‘accept/refuse’ packet and ‘transmission/monitor’ message between cluster heads and member nodes in this round. Moreover a number of keys

like ‘network’ key, ‘cluster’ key and ‘temporal pair wise’ key are used in the cluster round. In the primary security module dynamic authentication of new nodes is done with Tesla certificates and by exchanging pair wise key between neighbouring nodes and new nodes. The major advantage of this scheme is designing an intrusion detection module which is capable of detecting and eliminating attacks from outside and inside adversary nodes in the network. Though the authors have claimed that joining of new nodes is an additional feature of their scheme, which is not very significant as new nodes can join the network only at the end of cluster rounds and not at any time of network operation.

Authors in [13] have proposed two protocols for dynamic key establishment between member node and cluster head as cluster heads are changed periodically in cluster based architectures. These protocols provide for establishing secret key for secured communication between member nodes and cluster heads. The merit of the work is, during dynamic key establishment, no additional communication overhead is incurred that makes it energy-efficient. Although the scheme considers outside attackers who is able to eavesdrop on the communications, no mechanism is developed to secure communication between CH and BS.

Further, a number of contemporary works dealing with security in wireless adhoc network are reported and the works do not use key based authentication. Kao and Marculescu have proposed a couple of such securing strategies. In one of their works [14] they have proposed an attack prevention mechanism by minimizing eavesdropping risk where the risk minimization is done through transmission power control of the node. The power control has been employed in such a way that transmission range is adapted to reduce sniffing. The strength of the paper lies on its simplicity where, unlike high-overhead cryptographic solution, security is provided by minimizing eavesdropping risk controlling transmission power of the nodes. In their other work [15] they have proposed an anonymous multicast algorithm which is also not on key based authentication. To make it secure this algorithm instead of exchanging vulnerable route details predicts the details based on the statistical properties of the network. The usage of such statistical properties instead of actual information makes the scheme energy-efficient which is identified as major contribution of the paper. In another work [16] unlike the common security handling mechanisms provided from upper layers (e.g. key management), the security solution is given at physical layer. This work also employs power control mechanism. Here in addition to reducing the transmission radius of the transmitter, it allows unprotected links to play a more active and flexible role to jam the eavesdropper receiver while improving quality of the secure link. The commonality among these works [14–16] is, the works do not employ key

management mechanism for securing the wireless networks. These solutions should not be considered as alternative to the key based authentication solution. The two sets of work [7, 9–13] and [14–16] should instead be considered as complimentary to each other. The presence of representative solution from either set provides additional enhancement of wireless network security against eavesdropping.

All the schemes described above implement secured cluster head-base station and/or cluster head-member node communication, exchanging information in the form of key, trust value, alarm packet etc. over the air. Therefore, irrespective of degree of intricacies of the securing techniques, the probability of capturing this information by adversaries is non-zero. This inspires us to devise a securing scheme for WSN where question of exchanging such information over unguided medium does not arise and thereby ensures absolute security. The objective of our work is to propose a light-weight scheme for securing query processing in wireless sensor network. The proposed security scheme gives a solution of replay attack while ensuring basic security in terms of authentication, data integrity and data freshness.

The rest of the paper is organized as follows. In Sect. 2, system model along with a brief description of the present work is given. Section 3 provides a detailed description of the scheme. Performance analysis of the scheme along with a detailed comparative study is made in Sect. 4. In Sect. 5, concluding remarks highlighting future scope of the work are presented.

2 System model

The system model in the present work considers clustered network architecture for a query processing domain in WSNs. A single 16-bit key is preloaded into all the nodes of the network prior to deployment. This 16-bit pre-deployed key is a single network wide key [17]. The advantage of this key management includes minimal storage requirements and avoidance of complex protocols. Only a single key is to be stored in the nodes’ memory and once deployed in the network, there is no need for a node to perform key discovery or key exchange since all the nodes in communication range can transfer messages using the key which they already share.

Once the key deployment is over, nodes organize themselves into local clusters with one node acting as the cluster head (CH). The clusters are formed following LEACH protocol [6] which consists of the phases- advertisement phase, cluster set-up phase, schedule creation phase and data transmission phase. In cluster set-up phase nodes send messages for joining a cluster to the respective

cluster heads. Once a CH accepts a node as a member of the cluster, it sends the acceptance along with a 4-bit schedule-id (assigned as per the acceptance order) of the member node. For example, in cluster set-up phase, the cluster assigns 0 as schedule-id to the first accepted member node, assigns 1 as to the second accepted member node and so on.

On completion of the advertisement and cluster set-up phases, the schedule creation phase starts. The TDMA schedule creation by the CH depends on the number of its MNs. According to the TDMA schedule, the MNs communicate with the CH. The TDMA schedule is broadcast towards the MNs by the CH in encrypted form. For implementing secured communication of the TDMA schedule between CH and MN, the nodes use the 16-bit keys pre-deployed into them. All the CHs use this key only to ensure authenticated transmission of TDMA schedule whereas all the MNs use this key to verify authenticity of the broadcast TDMA schedule. The mechanism provided for secured broadcast of the TDMA schedule is presented in Sect. 2.3.

At the end of schedule creation phase, the data transmission phase starts when all the MNs under a CH are capable of computing their respective slots of the TDMA frame for communication in this phase. The computation of slots in data transmission phase is elaborated in Sects. 3.2 and 3.3. The structure of TDMA frame for data transmission phase is shown in Fig. 1. The slots are for three purposes—one is listening slot when MNs listen to the broadcast messages from the CH, the other is communication slot when unicast communication takes place between MN and CH and the last slot is the aggregation slot used by the CH for aggregating data received from the MNs after the end of the communication slot.

The CHs directly communicate with the base station/sink to reach the external world. LEACH performs local data fusion [18] at CHs to reduce the amount of data sent from the clusters to the external world via the base station. Data fusion also known as data aggregation is a technique for combining data from different sources enroute to the destination in order to remove redundancy. This technique is used in a variety of applications in WSN. For example, to calculate the maximum humidity of a particular area, the readings from the sensor nodes in a cluster are combined at the cluster head using a specific aggregation function e.g. MAX and the result is sent to the base station.

The communication between a member node (MN) and its cluster head follows time division multiple access (TDMA) whereas the communication between a cluster head and the base station follows code division multiple access (CDMA). However, in data transfer phase, communication between MN and CH follows dynamic TDMA which is a variant of TDMA. The present scheme

authenticates member nodes in the form of registering the nodes before actual query processing starts. As the CDMA codes are unique for all the CHs, explicit authentication of cluster heads is not necessary.

The present work considers a query-driven application platform where base station generates query messages and broadcasts the query towards the CHs. The CHs receive the broadcast query which they further broadcast towards their MNs. However, before the CHs broadcast query toward the MNs, they start registering their respective member nodes for authentication. During registration, if an MN is authenticated, the CH sends a node-id to the MN. Once the registration is over, the MNs successfully authenticated are considered as registered. Now the CH broadcasts the query message towards the MNs. In response to the query, MNs send their query responses to the CH. On receiving the query responses from MNs, i.e. when the time assigned for the TDMA schedule of the MNs gets over, the CH aggregates the data in the aggregation slot (Fig. 1) and forwards it to the base station. The aggregation, as mentioned earlier, is done employing a suitable aggregate function (e.g. max, min, avg.) depending on the query of the application domain.

2.1 Assumptions

In this work we consider static sensor network with homogeneous nodes. As homogeneous nodes are considered, therefore they are equal in terms of storage, communication and computational capabilities. The nodes have a pre-deployed 16-bit key with them which is used for authenticating the TDMA schedule received from the CH. This pre-deployed key is not used further for implementing the proposed query processing scheme.

Our architectural model divides the network into clusters similar to the cluster formation mechanism described in [6]. Every cluster contains a cluster head along with its member nodes. We assume that no new nodes join the network except the nodes which are present at deployment time. Communication in the network follows- base station broadcasts queries towards cluster heads which in turn broadcast these queries toward member nodes.

2.2 Attack

An attack [8] is defined as an attempt to gain unauthorized access to a service, resource, or information, or the attempt to compromise integrity, availability, or confidentiality. Denial of Service (DoS) is one type of attack, which hinders communication among the sensor nodes. The present work considers replay attack, which is one form of path based DoS (PDoS) attack. If there is a PDoS attack, the network gets flooded with bogus packets along a multi-hop

delivery path [19]. The occurrence of bogus packets in the network is due to the replaying of packets by adversaries leading to replay attack. So PDoS attacks in the network may result in replay attacks. Alternatively, replay attack is an attack where packets are captured and replayed into the network by an adversary [20]. It can threaten the normal working condition of WSN in general and clustered WSN in particular. In clustered WSN, if this attack is launched, it can keep the CHs busy by replaying packets and thereby the network may stop functioning either partially or fully.

The attacker considered in this work is mote class attackers which have the same capabilities as ordinary sensor nodes and are not able to launch Brute Force attacks.

2.3 Secured schedule creation

The TDMA schedule packet broadcast by the CH towards its MNs contains a 2-tuple (32-bit encrypted 'T', 8-bit ' Δ ') where T is the starting time-stamp of the TDMA schedule and Δ is slot duration. The CH encrypts T as follows:

Pad T with four zeroes

Padded T $\oplus$ k = T'

Arrange alternately T and T' (32-bit encrypted T)

where k is 16-bit pre-deployed key. Now the encrypted TDMA schedule broadcast packet is also a 2-tuple (32-bit encrypted T, 8-bit slot duration).

Once an MN receives the encrypted TDMA schedule, it decrypts the schedule as follows:

Retrieve T and T' from encrypted T

$T \oplus T' = k''$

The MNs store T and check whether $k'' = k$. If the result of check is affirmative, the MN considers that the received TDMA schedule is authentic, otherwise it discards the TDMA schedule packet. Upon receiving authentic schedule,

an MN is now equipped with (Δ , schedule-id) with the help of which it computes its starting time-slot for communication according to the requirement of two different phases (Sects. 3.2 and 3.3) of the proposed query processing scheme.

Check for replay attack

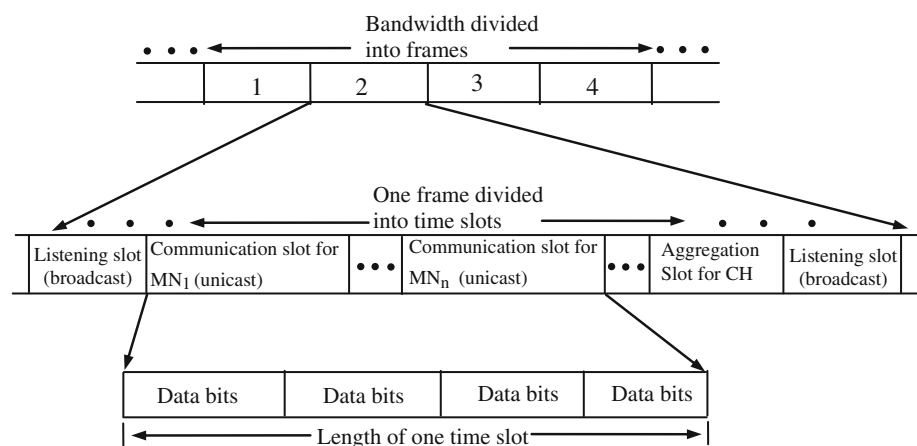
In case there is a replay attack, malicious node captures the TDMA schedule packet and tries to replay the same. This attack will be of use only in the next TDMA schedule creation phase. If a malicious node poses as CH and sends the schedule packet, the MNs check the contents of the replayed packet and compare the time-stamp (T) in the replayed packet with the time-stamp that they have stored from the previous schedule broadcast. If both the time-stamps are same, the MNs discard the TDMA schedule packet broadcast by the adversary.

2.4 Dynamic TDMA

Time Division Multiple Access (TDMA) is a bandwidth sharing method where nodes share the available bandwidth in time. As illustrated in Fig. 1, the available bandwidth is divided into frames and each frame is divided into time-slots. The number of time slots of the TDMA frame depends on the length of the frame. Data is transmitted in the form of bursts in the time slots. The number of data bits i.e. the length of the data bursts transmitted through a time slot depends on the length of the time-slot and the transmission bit rate of the system. To implement TDMA, proper synchronisation between the receiver and transmitter is essential. Guard bands [21] are introduced between time slots to prevent intersymbol interference and synchronisation errors.

One variant of TDMA is the dynamic TDMA [22–27] which again has two variants. In one variant of dynamic TDMA, scheduling algorithm dynamically reserves a variable number of time-slots in each frame to accommodate

Fig. 1 Structure of a TDMA frame



variable bit-rate data streams, based on the traffic demand of each data stream. It facilitates several application domains e.g. integrating voice and data on a single channel used in various technologies such as IEEE 802.16a Wi-Max, Bluetooth. In another variant of dynamic TDMA, the concept of mini-slots is introduced for dynamically reserving the timeslots. The available bandwidth is divided into frames and each frame is divided into time-slots. These time-slots are further divided into smaller slots known as mini-slots.

2.5 Objective

The objective of the proposed Secured Query Processing Scheme using dynamic TDMA (SQPS_DT) is to give a light-weight solution to ensure basic security in general and prevent replay attack in particular while working for an event driven query domain application. The essential properties [28] of a WSN required for maintaining basic security within the network are:

- **Authentication**—A sensor network must be able to differentiate between data coming from reliable sources and those coming from adversaries.
- **Data Integrity**—The received data should not be tampered during communication between the sender and the receiver.
- **Data Freshness**—The data received at a particular time should be current data and not old data which may be replayed by adversaries.

3 The proposed secured query processing scheme

In this secured query processing scheme, queries are periodically broadcast from base station. All the CHs under the BS receive the query. The CHs in turn broadcast the query toward their MNs. So the BS, CHs and MNs all are involved in query processing. In the context of securing the scheme, authentication of all the types of nodes are essential. However, as mentioned in Sect. 2, the communication between BS and CH follows CDMA where the CDMA codes are unique for all CHs. So, explicit authentication of cluster heads is not necessary. The scheme has two phases:

- Query phase
- Query response phase

The activities in query phase occur in two different places- one is between base station and cluster head and the other is between cluster head and member node.

- Query phase (BS–CH)
- Query phase (CH–MN)

The activities in query response phase also occur in two different places- one is between member node and cluster head and the other is between cluster head and base station.

- Query response phase (MN–CH)
- Query response phase (CH–BS)

3.1 Query phase (BS–CH)

This phase is used for authenticating the base station from which cluster heads are supposed to receive the broadcast queries. This phase also ensures that even if any malicious node tries to send bogus queries or same copy of an already sent query it is prevented from doing so.

Before sending a query the BS initiates the session by broadcasting the present time-stamp $T_{BS_present}$ (12 bits) to all CHs at time $(t-2)$. Both the BS and CH store $T_{BS_present}$. The BS then broadcasts the query in encrypted form at $(t-1)$.

3.1.1 Encryption at base station

The 8-bit query (Q) is encrypted in two steps. In the first step encryption is done as follows:

$$Q \oplus T_{BS_present} = Q'$$

The 8 bit query is appended with four 0s from the left to make it compatible for XORing with the 12 bit time-stamp. In the next step Q' is intermingled with Q by alternately arranging the bits of Q and Q' starting from the first bit of Q to form the encrypted query packet (Q''). Now the encrypted query packet is broadcast by the BS towards the CHs at time $(t-1)$.

3.1.2 Decryption at cluster head

After receiving the encrypted query packet, decryption is also done in two steps. In the first step the CHs retrieve Q and Q' from the encrypted query packet. This is done by alternately separating the bits of Q and Q' from the encrypted query packet i.e. the bits at MSB and at (MSB-1) of the query packet are to be equal to the MSB of Q and Q' respectively. In the next step, decryption is performed as follows:

$$Q \oplus Q' = (T_{BS_present})'$$

If $(T_{BS_present})'$ is equal to $T_{BS_present}$ stored in CH, it can be inferred that the BS is authentic.

3.1.3 Algorithm for Query Phase (BS–CH)

Begin

```

// Action executed by Base Station (BS)
1: broadcast  $T_{BS\_present}$  towards CHs at time (t-2) /*The BS initiates the session by broadcasting  $T_{BS\_present}$  to the CHs at time (t-2) */
2: store  $T_{BS\_present}$  /*  $T_{BS\_present}$  is stored by BS */
// Action executed by Cluster Heads (CHs)
3: store  $T_{BS\_present}$  /*  $T_{BS\_present}$  is also stored by CH */
// Action executed by BS
4: encrypt query (Q)
5:  $Q \oplus T_{BS\_present} = Q'$  /* The query Q is encrypted by XORing it with  $T_{BS\_present}$  */
6: Intermingle Q and Q' in alternate manner to get encrypted query packet (Q')
7: broadcast Q' at time (t-1) /* encrypted query packet is broadcast towards the CHs at time (t-1) */
// Action executed by CHs
8: retrieve Q and Q' from Q'
9: perform  $Q \oplus Q' = (T_{BS\_present})'$ 
10: if  $(T_{BS\_present})' = T_{BS\_present}$  then /* the result is compared with the stored  $T_{BS\_present}$  */
11:   accept query (Q) /* CH accepts the query packet and authenticates the BS */
12: else
13:   reject query /* CH rejects the query packet */
14: end if
End

```

3.2 Query phase (CH–MN)

This phase is used for authenticating the member nodes by their cluster heads. Authentication of member nodes is done by registering the nodes by the respective cluster head nodes. The objective of this phase is to register only the authentic nodes. Moreover this phase provides a mechanism that will help a CH to ensure data integrity and data freshness during query response phase. Upon receiving a query from the base station, the cluster head broadcast registration packet towards the MNs and sends the time ($T_{CH_present}$) of broadcasting of registration packet to base station as well as towards the MNs. In this phase every MN uses three time slots (Fig. 2) of the TDMA frame for sending registration response to the CH. On receiving the time of registration broadcasting and realizing it as starting of registration, MNs compute their actual starting slot for communication with the help of their respective schedule-id (Sect. 2) and Δ as follows: $T_{CH_present} + 2\Delta + \text{schedule-id} \times 3\Delta$.

The registration packet (S) contains 8 bits including '0' as MSB. This MSB differentiates between a registration packet and a query packet sent by the cluster heads. Each member node uses its designated TDMA time slot for sending a registration-response (RR) packet to the cluster head as well as for receiving its identification (node-id) generated by the cluster head after authenticating it. On receiving the registration packet, the respective member nodes generate key (B) for securing communication between the CH and MN. Subsequent to this, the MN

encrypts S to get S' . The key with which encryption of S is done to obtain S' is sent in the RR packet in encrypted format (K). Thus a 3-tuple ($K, S', T_{MN_present}$) RR packet is created by the MN which is sent to the CH.

When a CH broadcasts registration packet towards its MNs, due to several reasons all the MNs may not receive the broadcast packet. So only the MNs, which have received registration packet successfully send RR packet to CH. In WSN, data redundancy is a desirable feature because of the unreliable nature of wireless medium. This redundancy is achieved by deploying redundant nodes. So even if a few nodes (30%) fail to receive registration packet, it does not affect the operation of the network. In the worst case, if the number of RR packets received at CH is below a threshold (70% of respondent nodes), the CH discards those RR packets i.e. this cluster would not participate in query processing. If this continues in the next communication round when another query is being sent, the CH assumes that some problem has occurred in the MNs e.g. drainage of battery power and the cluster formation process is to be repeated. Here we consider query phase followed by query response phase makes a communication round.

An RR packet is logically split into two parts. The length of the first part, starting from MSB of RR packet, is equal to the total length of K and S' where K is produced by encrypting B at the member node; the length of the second part is equal to the length of the rest of the RR packet. Similarly three time-slots used by an MN are also

split into two logical slots referred as first and second mini-slots. The first part of the RR packet is sent in the first mini-slot and the second part of RR packet is sent in the second mini-slot. In between the two mini-slots a time gap equal to the time required to transmit 1 bit of data is kept as gap slot. This time gap helps the cluster head to distinguish between the two mini-slots.

Once the CH receives RR packet, it retrieves S' (8-bit from LSB) and K (rest of the bits) from the data received in first mini-slot. Next it computes the key (B) by decrypting K and gets back the original registration packet (S) by decrypting S' with the key B . If the result of decryption of S' is equal to the original registration packet, the corresponding MN is authentic and the CH sends a node-id to the MN in the same mini-slot as used by the MN for sending the remaining bits of RR.

3.2.1 Data stored at base station

As mentioned earlier, a CH sends the time of broadcasting of registration packet to BS immediately before it starts registering member nodes. Accordingly the BS stores the time-stamp $T_{CH_present}$ and an additional time-stamp $T_{CH_previous}$ for future use which is set to null initially.

3.2.2 Encryption at member node

On receiving the registration packet S , member nodes randomly choose the key (B). Now S is encrypted by left-shifting B -bit to get S' . For example if $S = m_7 \dots m_0$, where MSB is m_7 and LSB is m_0 and B is 100, S is encrypted by left shifting B -bit resulting $m_3 m_2 m_1 m_0 m_7 m_6 m_5 m_4$ (S'). Also B is encrypted and the encrypted value (K) is sent in the RR packet. The encryption is done as follows: $K = B \oplus T''$ where $T'' =$ Number of bits equal to the value of schedule-id (Sect. 2) starting from MSB of $T_{MN_present}$. If length of $B >$ length of T'' , pad ($B - T''$) number of zeroes with T'' from the MSB or if $T'' > B$, pad ($T'' - B$) number of zeroes with B from the MSB. For example, when

$T_{MN_present} = 001101100011$, $B = 100$ and schedule-id of the node = 0101, T'' is equal to the first 5 bits of $T_{MN_present}$ starting from MSB i.e. $T'' = 00110$. As length of $T'' >$ length of B , B is padded with $(5(T'') - 3(B)) = 2$ zero bits from MSB. So the padded $B = 00100$. The encrypted key value $K = B \oplus T'' = 00100 \oplus 00110 = 00010$. Accordingly a 3-tuple $(K, S', T_{MN_present})$ RR packet having 32 (12 (Max.) + 8 + 12) bits size is formed by the member node where $T_{MN_present}$ indicates the present time-stamp at which the RR packet is being sent.

Figure 2 shows the time slots of a dynamic TDMA frame used by MNs in the Query phase (CH–MN) for the above example. This frame consists of $3n+1$ time slots where each of the slots communicates 13 data bits. As shown in Fig. 1, the first slot TS_1 is listening slot and TS_2 to TS_{3n+1} are communication slots for n number of MNs. In listening slot, the MNs listen to the broadcast of registration packet by the CH and the remaining $3n$ communication slots are used for communication between the CH and the MNs. Every 3 slots of these $3n$ slots are divided into 2 mini-slots [25] with a gap slot to differentiate between the two mini-slots. Figure 2 also shows how three slots are split logically to two mini-slots. The 2nd mini-slot is shared by an MN and its CH.

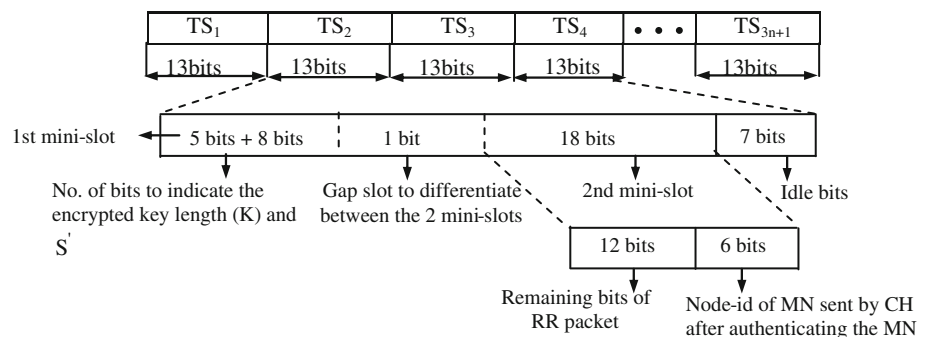
3.2.3 Transmission of registration-response (RR) packet

The RR packet is transmitted using two mini-slots (Fig. 2) to prevent malicious nodes from obtaining the value of key used by MNs even if the malicious node captures the RR packet. The gap slot used between two mini-slots is provided to aid the CH to differentiate the two splits of the RR packet.

3.2.4 Decryption at cluster head

Upon receiving the RR packet $(K, S', T_{MN_present})$ from an MN, the CH concatenates the data received in two mini-slots, decrypts the encrypted key length K to get the key B and retrieves the registration packet (S) by right shifting it

Fig. 2 Example of TDMA frame used in Registration



by B bit. The decryption of key is as follows- $T'' \oplus K = B$. The CH considers the length of the packet received in first mini-slot as the total length of K and S' . So S' is right shifted B times and converted to S'' . After decrypting, if S'' matches with S, the CH accepts the registration packet and authenticates the corresponding member nodes.

If the CH accepts S, then it sends the node-id consisting of 6 bits to the MN in the remaining time of the 2nd mini-slot. Otherwise the left over time of the 2nd mini-slot remains idle.

3.2.5 Data stored at cluster head

Once the CH authenticates an MN, it generates an identification (node-id) for the MN and stores a 7-tuple data (node-id, schedule-id, S, S' , K, $T_{MN_present}$, $T_{MN_previous}$) for the MN where (S' , K, $T_{MN_present}$) is RR (packet sent by an MN designated by the node-id), K is the encrypted key value, S is the original registration packet, schedule-id (Sect. 2) and $T_{MN_previous}$ is a time-stamp which is set to

null initially. The size of the data for each authentic MN stored at CH is 62 (6 + 4 + 8 + 8 + 12 + 12 + 12) bits $\approx$ 8 bytes. The CH also stores $T_{CH_present}$ which is the registration packet broadcast time.

3.2.6 Data stored at member node

Once an MN gets its node-id from the CH in the Query phase (CH–MN), the MN stores 5-tuple (node-id, schedule-id, Δ , K, $T_{MN_present}$) data where $T_{MN_present}$ is time-stamp with which the MN is authenticated and gets its node-id. The size of the data stored at an MN for getting the MN authenticated is 42 (6 + 4 + 8 + 12 + 12) bits $\approx$ 6 bytes.

3.2.7 Algorithm for query phase (CH–MN)

Whenever the BS broadcasts a query towards CHs, the CHs start registering MNs, the first phase of query processing, as follows.

Begin

// Action executed by cluster heads (CHs)

1: send $T_{CH_present}$ to base station and MNs

2: store $T_{CH_present}$

// Action executed by base station (BS)

3: store 2-tuple data

// Actions executed by member nodes (MNs)

4: compute starting slot = $T_{CH_present} + 2\Delta + \text{schedule-id}_i \times 3\Delta$

// Action executed by CHs

5: broadcast S towards MNs at time t

// Actions executed by member nodes (MNs)

6: On receiving S at time (t+1)

7: for $i=t+1, i \leq t+3n+1, i++$

8: generate B_i

9: perform B-bit left shift on S to obtain S'_i

10: compute T_i'' from $T_{MN_present}$ extracting number of bits equal to the schedule-id of MN_i starting from MSB

11: compute $K_i = B_i \oplus T_i''$

12: generate 3-tuple RR_i

13: compute two mini-slots $(t+1)_1$ & $(t+1)_2$

14: split RR_i into $(RR_i)_1$ & $(RR_i)_2$

15: if $(t+1) = (t+1)_1$ then

16: send $(RR_i)_1$

17: else

18: if $(t+1) = (t+1)_2$ then

19: send $(RR_i)_2$

20: else

21: delay 1 bit

22: end if

// Actions executed by CH on receiving $(RR_i)_1$ & $(RR_i)_2$

/* $T_{CH_present}$ is the registration packet broadcast time */

/* CH stores $T_{CH_present}$ */

/* $T_{CH_present}$, $T_{CH_previous}$ */

/* MNs compute their respective starting slot of the TDMA schedule */

/* CH broadcasts Registration packet (S) */

/* MN_i generates the key B_i */

/* MN_i encrypts S by left shifting it by B_i bit to obtain S'_i */

/* T_i'' is obtained from $T_{MN_present}$ */

/* The key is encrypted by XORing B_i and T_i'' */

/* MN_i generates RR_i (K_i , S'_i , $T_{MN_present}$) */

/* 1st & 2nd mini-slots */

/* splitting is based on K_i and S'_i */

/* MN_i sends (length K_i + length S'_i) number of bits from MSB of RR_i */

/* MN_i sends remaining $((40 - (K_i + S_i)) - 1)$ bits of RR_i */

/* 1 bit gap slot */

```

23: extract (length of  $((RR_i)_1 - 8)$ ) number of bits
    from  $(RR_i)_1$  starting from MSB to get  $K_i$ 
24: compute  $T_i''$  from  $T_{MN\_present}$  extracting          /*  $T_i''$  is obtained from  $T_{MN\_present}$  */
    number of bits equal to the schedule-id
    of  $MN_i$  starting from MSB
25: compute  $B_i = T_i'' \oplus K_i$                           /* CH performs decryption to obtain  $B_i$  */
26: perform B-bit right shift on  $S_i'$  to obtain  $S_i''$       /* CH decrypts  $S_i'$  by right shifting  $B_i$  bit to obtain  $S_i''$  */
27: check for authenticity of  $RR_i$                       /* CH checks authenticity of  $RR_i$  by comparing  $S_i''$  with  $S_i'$  */
28:   if  $S_i'' = S$  then
29:     accept  $RR_i$                                        /* CH accepts registration-response packet  $RR_i$  and authenticates  $MN_i$  */
30:     generate node-id $_i$  for  $MN_i$ 
31:     send node-id $_i$  to  $MN_i$ 
32:     store 7-tuple data                                /* 7-tuple data: (node-id $_i$ , schedule-id $_i$ ,  $S$ ,  $S_i'$ ,  $K_i$ ,  $T_{MN\_present}$ ,  $T_{MN\_previous}$ ) */
33:   else
34:     reject  $RR_i$                                        /* CH rejects  $RR_i$  and  $MN_i$  */
35:   end if
// Action executed by  $MN_i$  on receiving node-id $_i$ 
36:     store 5-tuple data                                /* 5-tuple data: node-id $_i$ , schedule-id $_i$ ,  $\Delta$ ,  $K_i$ ,  $T_{MN\_present}$  */
37: end for
// Actions executed by CH before performing data aggregation
38: if  $(RR_i)_1$  &  $(RR_i)_2$  received from < 70% MNs      /* CH checks whether RR received from < 70% MNs */
39:   discard packets received from MNs                  /* CH discards packets received from MNs */
40: end if
End

```

Authenticating nodes is one of the prime importances with respect to security for any network. The Query phase (CH–MN) of the present scheme for registering the member nodes ensures that the nodes present in the network are authenticated. The same key is used for encryption and decryption of the data packets and therefore it is a symmetric key cryptography.

In a cryptographic security solution, key is a major parameter of concern. The key used in the network is either distributed among the nodes prior to deployment or is exchanged among the nodes through packet transmission [10, 11]. Our query processing scheme exploits the second way of getting the key by the node. However, the key is not directly exchanged among the nodes; rather the encrypted version of key information is exchanged thereby preventing malicious nodes from getting the key even if it gets hold of any packet being exchanged during authentication of the nodes. In a word, there is zero probability for any malicious node to decipher the key from the captured packet during authentication.

3.3 Query response phase (MN–CH)

In the event of query processing, the base station broadcasts query. Upon receiving the query, a CH starts registering its member nodes i.e. Query phase (CH–MN) starts. Once this phase is over (Sect. 3.2), the CH broadcasts the query as well as time (T_{QB}) of query broadcasting towards

its member nodes. Query packet is of 8 bits having MSB 1 which distinguishes it from a registration packet. Each of the MNs sending query response utilizes four time slots. Upon receiving T_{QB} and realizing it as a query response phase, the MNs compute their actual starting slot for sending query response to the CH as follows: $T_{QB} + 2\Delta + \text{schedule-id} \times 4\Delta$. In response to the query, MNs send reply of the query by sending query response (QR) packet. The MNs (if any) which have no responses to send remain silent during their allocated time-slots.

3.3.1 Encryption at member node

On receiving a query packet, MNs prepare response (m) of the query. Now to encrypt m , the key (B) is computed as follows: $B = K \oplus T''$ where T'' is computed extracting number of bits equal to the schedule-id of the MN from $T_{MN_present}$. Then m is encrypted by right-shifting B-bit to obtain m' . Subsequent to this, a query response (QR) packet is prepared which is a 5-tuple (node-id, m , m' , $T_{MN_present}$, $T_{MN_previous}$) data. Here $T_{MN_present}$ is the present time-stamp when the response packet for the query is being sent by the MN and $T_{MN_previous}$ is the time-stamp of last communication between the MN and CH. Here the last communication means the time-stamp ($T_{MN_present}$) stored by the MN at Query phase (CH–MN). The size of the QR packet is 46 bits—node-id (6), m (8), m' (8), $T_{MN_present}$ (12) and $T_{MN_previous}$ (12).

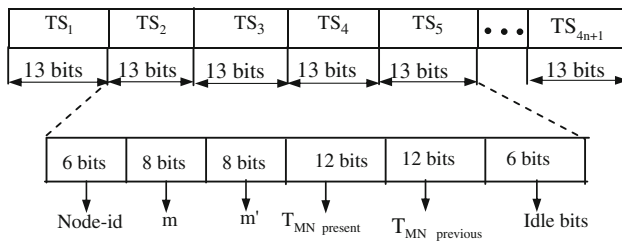


Fig. 3 Time slot allocation for QR (MN-CH) communication

The structure of the TDMA frame used in the query response phase between cluster head and member node is shown in Fig. 3. The time slot TS_1 of this frame is used by a CH for broadcasting the query. For n number of MNs in a cluster, the total number of time slots required for providing four time-slots to each of the MNs is $4n$ and therefore TS_2 to TS_{4n+1} slots are used by the MNs. As 4 time-slots are required to accommodate the QR packet of 46 bits, so the slot duration equivalent to transmit 6 ($13 \times 4 - 46$) bits of data would remain unutilized and are referred to as idle bits. Further, after all the n members' slots for sending query response are over, there is an aggregation slot as shown in Fig. 1. The CH aggregates all the received query responses applying relevant aggregation function depending on the requirement of application domain.

3.3.2 Data stored at cluster head

After receiving query response packet from an MN, the CH updates the 7-tuple data stored at query phase (CH-MN). In the query phase (CH-MN) the stored 7-tuple data was (node-id, schedule-id, $S, S', K, T_{MN_present}, T_{MN_previous}$). In the query response phase (MN-CH), S and S' attributes are replaced by m and m' respectively. $T_{MN_previous}$ which contained null value during query phase (CH-MN) are now replaced by $T_{MN_present}$ which was stored during Query

phase (CH-MN) and $T_{MN_present}$ is replaced by the time when the query response packet is being sent by the member node. The size of the data for query response from each authentic MN stored at CH is 62 ($6 + 4 + 8 + 8 + 12 + 12 + 12$) bits ≈ 8 bytes.

3.3.3 Decryption at cluster head

On receiving the QR packet the CH finds the node-id of the MN from the packet. It computes the key (B) for decryption as follows- $B = K \oplus T''$. Now the encrypted m' is decrypted by left-shifting B -bit to obtain m'' . Once the CH decrypts the message part of the QR packet, it checks for data integrity, data freshness and replay attack.

3.3.4 Check for data integrity

If it is found that the decrypted message m'' is same as the original message m , data integrity is preserved.

3.3.5 Check for replay attack and data freshness

In replay attack the malicious node repeats the already sent packets and results in energy exhaustion of nodes and eventual collapse of the network. With the help of two attributes $T_{MN_present}$ and $T_{MN_previous}$ stored at CH corresponding to every registered/authenticated MNs, replay attack is detected. The CH compares $T_{MN_present}$ stored in it and the $T_{MN_previous}$ in QR and if these are found equal, then it can be ensured that no replay attack has taken place and data freshness is preserved.

3.3.6 Algorithm for query response phase (MN-CH)

Begin

// Action executed by CH

1: Broadcasts query packet received from BS at time $(t+3n+2)$

// Actions executed by member nodes (MNs)

2: On receiving query packet at time $(t+3n+2)$

3: for $i = t+3n+3, i \leq t+7n+3, i++$

4: compute starting slot = $T_{QB} + 2\Delta + \text{schedule-id}_i \times 4\Delta$

/* MN_i computes its starting time for communication */

5: compute T_i'' from $T_{MN_i_present}$ extracting number of bits equal to the schedule-id of MN_i starting from MSB

/* T_i'' is obtained from $T_{MN_i_present}$ */

6: compute $B_i = T_i'' \oplus K_i$

/* MN_i retrieves B_i for encrypting m_i */

7: perform B_i -bit right shift on m_i to get m_i'

8: generate 5-tuple QR_{MN_i-CH}

9: send 5-tuple QR_{MN_i-CH} to CH

/* i^{th} member node sends query response packet, QR_{MN_i-CH} (node-id, m_i, m_i' (m_i right shifted by B_i bit)), $T_{MN_i_present}, T_{MN_i_previous}$) */

// Action executed by CH on receiving QR_{MN_i-CH}

10: compute T_i'' from $T_{MN_i_present}$ extracting number of bits equal to the schedule-id of MN_i starting from MSB

/* T_i'' is obtained from $T_{MN_i_present}$ */

```

11: compute  $B_i = T_i'' \oplus K_i$  /* CH performs decryption to obtain  $B_i$  */
12: perform  $B_i$ -bit left shift on  $m_i'$  to obtain  $m_i''$  /* CH decrypts  $m_i'$  by left shifting it by  $B_i$  bit to obtain  $m_i''$  */
13: if  $m_i'' = m_i$  then /* check for data integrity */
14:     accept  $QR_{MN_i-CH}$  /* data integrity is maintained */
15: else
16:     reject  $QR_{MN_i-CH}$  /* data integrity is violated */
17: end if
18: if  $CH.T_{MN_i\_present} = QR_{MN_i-CH}.T_{MN_i\_previous}$  then /* CH compares  $T_{MN_i\_present}$  stored in it with  $T_{MN_i\_previous}$  in  $QR_{MN_i-CH}$  */
19:     accept  $QR_{MN_i-CH}$  /* no replay attack is detected */
20:     update its (CH's) stored data /* replaces  $S_i$  by  $m_i$ ,  $S_i'$  by  $m_i'$ ,  $T_{MN_i\_previous}$  corresponding to  $MN_i$  by  $T_{MN_i\_present}$ ,  $T_{MN_i\_present}$  by  $T_{MN_i\_present}$  of  $QR_{MN_i-CH}$  */
21: else
22:     reject  $QR_{MN_i-CH}$  /* CH rejects query response packet  $QR_{MN_i-CH}$  */
23: end if
24: end for
25: if  $QR_{MN_i-CH}$  received from  $< 70\%$  MNs /* CH checks whether  $QR_{MN_i-CH}$  received from  $< 70\%$  MNs */
26:     discard responses received from the MNs /* CH discards the responses received from the MNs */
27: end if
End

```

3.4 Query response phase (CH–BS)

On receiving query responses from the MNs, the cluster head aggregates the data. The aggregation is done by retrieving value of m from the records corresponding to every node-id and applying aggregate function depending on the requirement of the application domain. Now a 3-tuple ($m_aggregate$, $T_{CH_present}$, $T_{CH_previous}$) data is formed at CH as query response from CH to BS (QR_{CH-BS}) where $T_{CH_present}$ is the present time-stamp and $T_{CH_previous}$

3.4.1 Check for replay attack and data freshness

With the help of two attributes $T_{CH_present}$ and $T_{CH_previous}$ stored at BS corresponding to each CH, replay attack is detected. The BS compares $T_{CH_present}$ stored in it and the $T_{CH_previous}$ in QR_{CH-BS} and if these are found equal, then it can be ensured that no replay attack has taken place and data freshness is preserved.

3.4.2 Algorithm for query response phase (CH–BS)

```

Begin
// Action executed by cluster head (CH)
1: generate  $QR_{CH-BS}$  /* Query response is generated from the received  $QR_{MN-CH}$  packets */
2: send  $QR_{CH-BS}$  to base station (BS) /*  $QR_{CH-BS}$  is a 3-tuple data- ( $m\_arregate$ ,  $T_{CH\_previous}$ ,  $T_{CH\_present}$ ) */
// Action executed by base station (BS)
3: if  $BS.T_{CH\_present} = QR_{CH-BS}.T_{CH\_previous}$  then
4:     accept  $QR_{CH-BS}$  /* no replay attack is detected */
5: else
6:     reject  $QR_{CH-BS}$  /* CH rejects query response packet  $QR_{CH-BS}$  */
7: end if
End

```

is replaced by $T_{CH_present}$ stored at CH during Query phase (CH–MN).

Base station, on receiving the response from the cluster head, verifies whether the received response is recent i.e. it checks for replay attack in CH–BS communication. If no replay attack is found, the base station accepts the packet; otherwise the packet is rejected. If the base station accepts the packet, necessary update is performed on the data (Sect. 3.2.1) at BS— $T_{CH_previous}$ is now replaced by $T_{CH_present}$ stored in the BS and $T_{CH_present}$ is replaced by $T_{CH_present}$ in QR_{CH-BS} .

3.5 Diagrammatic representation of the scheme

The communication among different components of a wireless sensor network and control flow among various computational modules within such components of the network is shown in the activity diagram. Broadcast communication is shown by lightning symbol whereas unicast communication and control flows are shown by solid lines with arrowhead. All communication symbols are labeled to show the content of the packet along with the time-stamp at which it is being sent.

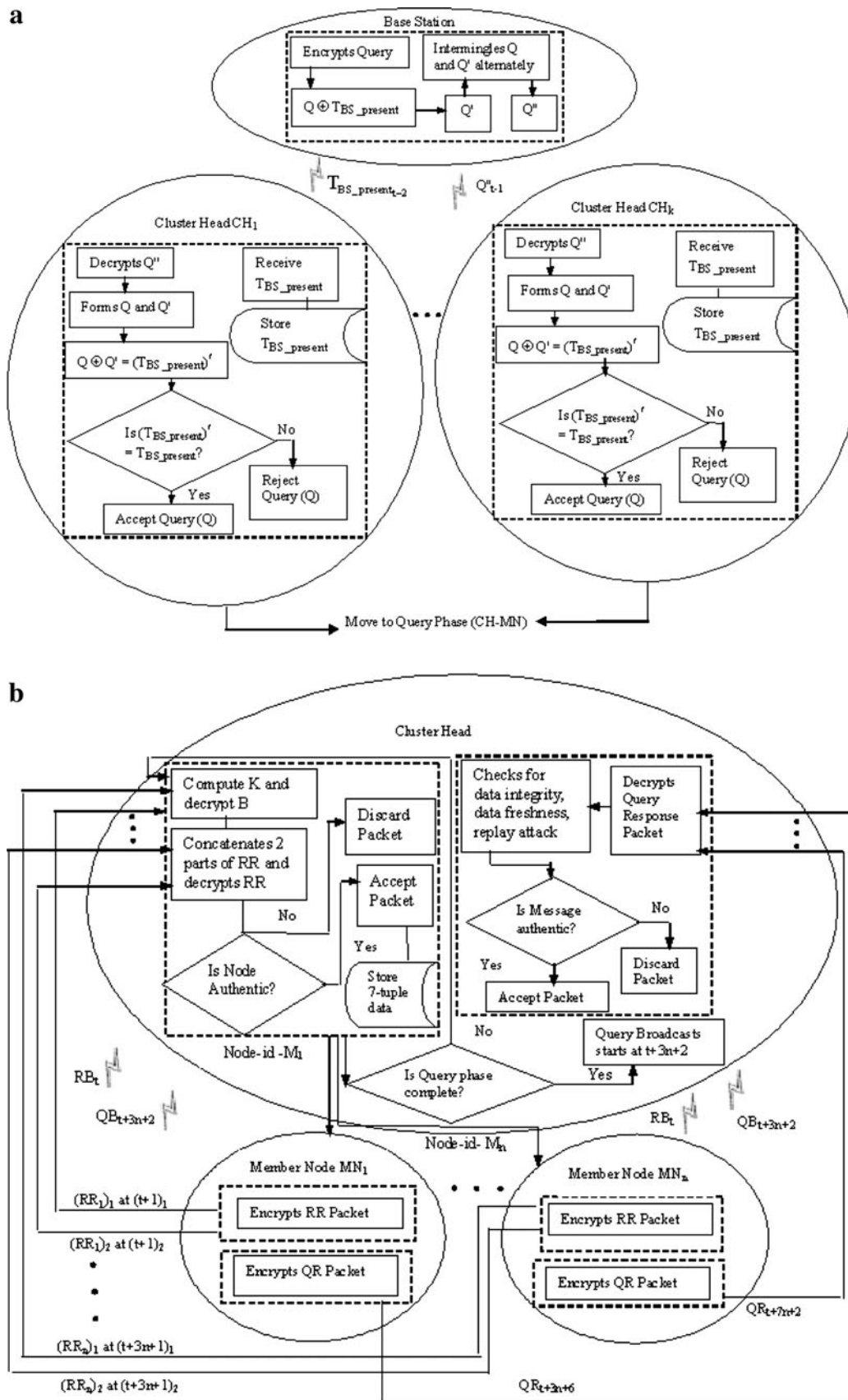


Fig. 4 **a** Activity diagram of query phase (BS-CH). **b** Activity diagram of query phase (CH-MN)

The communication from the base station to cluster head is depicted in Fig. 4 (a). The diagram shows how the query is encrypted by the base station and broadcast to the cluster heads at time (t-1). Also the decryption of the query packet by the cluster head is presented in the diagram. The broadcast communication labeled by $T_{BS_present, t-2}$ indicates that the present time-stamp is broadcast by the base station at time (t-2) and Q''_{t-1} Query Packet is broadcast by base station towards the cluster heads.

The communication between cluster head and member nodes is illustrated with the help of an activity diagram shown in Fig. 4 (b). The diagram shows how a cluster with n number of nodes and one cluster head works. The notations used in Fig. 4 (b) are described below:

RB_t	Registration Packet broadcast by cluster head at time t
$(RR_n)_1$ at $(t + n)_1$	Part of Registration-response packet transmitted by member node MN_n at time $(t + n)_1$
$(RR_n)_2$ at $(t + n)_2$	Part of Registration-response packet transmitted by member node MN_n at time $(t + n)_2$
Node-id- M_n	Node-id of member node MN_n transmitted by cluster head
QB_{t+3n+2}	Query packet broadcast by cluster head at time $t + 3n + 2$
QR_{t+3n+6}	Query response packet transmitted by member node MN_1 at time $t + 3n + 6$
QR_{t+7n+2}	Query response packet transmitted by member node MN_n at time $t + 7n + 2$

4 Performance analysis

The following section analyses the performance of the scheme SQPS_DT in terms of storage overhead, computation overhead and communication overhead. Analysis is performed considering mica2 motes [29] as the sensor nodes.

SQPS_DT is an extended version of our earlier scheme SQPS [30]. A comparative study is done among the schemes SecCBSN [12], SQPS and our proposed scheme SQPS_DT. Thorough security analysis comparing all the three schemes is also provided at the end of this section.

4.1 Storage overhead

In query phase (BS-CH) (Sect. 3.1), CH stores 12 bits $T_{BS_present}$ i.e. approximately 2 bytes of data.

In query phase (CH-MN), cluster head (CH) stores (Sect. 3.2.5) 7-tuple data (8 bytes approx.) for every member node (MN). So the CH stores $8n$ bytes considering n number of MNs associated with it.

Similar to the query phase (CH-MN), in query response phase also the same $8n$ bytes of data remain stored (Sect. 3.3.2) at CH after update. In addition to this it stores pre-deployed key of 16 bits (Sect. 2). Therefore, total storage overhead at CH = $(8n + 2)$ bytes = $(8n + 2)$ bytes.

On the other hand, in query phase (CH-MN), an MN, on receiving its node-id from the CH, stores (Sect. 3.2.6) 5-tuple data (size 6 bytes approx.). Also the MN stores 16-bit pre-deployed key and the starting time stamp T (16 bits) of the TDMA schedule. Therefore, total storage overhead at MNs = $(6 + 2 + 2)n$ bytes = $10n$ bytes.

4.2 Computational overhead

Secured TDMA schedule broadcast

In this phase computational overhead is incurred by both the CH and MNs for the following two tasks:

- XOR operation
- Alternate arrangement of 4 bytes of data

Each of the above tasks requires execution of 1 instruction which takes $0.125 \mu s$. So for authenticating the TDMA schedule between one CH and an MN, required computation time:

$$(0.125 \times 1 + 0.125 \times 4) \times 2 \mu s = (0.625 + 0.625) \mu s = 1.25 \mu s. \quad (1)$$

Query phase (BS-CH)

Similar to the schedule broadcast phase, in this phase the CH performs two tasks – one is for XOR operation and the other is for alternate arrangement of 3 bytes data. So computation overhead for this phase is:

$$(0.125 \times 1 + 0.125 \times 3) \mu s = 0.5 \mu s. \quad (2)$$

Query phase (CH-MN)

In this phase, computation overhead is incurred by an MN for the following tasks:

- Three multiplication operation
- Two addition operation
- Eight Left-shift operation
- One XOR operation
- Extraction operation.

Hence computation overhead of this phase for an MN is:

$$(0.125 \times 3 + 0.125 \times 2 + 0.125 \times 8 + 0.125 + 0.125) \mu s = 1.875 \mu s.$$

In the same phase, computation overhead is incurred by a CH for the following tasks:

Extraction operation
XOR operation
Right-shift operation
Comparison operation.

So computation overhead of this phase for a CH is:
 $(0.125 + 0.125 + 0.125 \times 8 + 0.125)\mu s = 1.375 \mu s$.

Therefore, query phase (CH–MN) incurs following computational overhead

$$(1.875 + 1.375)\mu s = 3.250 \mu s. \quad (3)$$

Query response phase (MN–CH)

In this phase, computation overhead is incurred by an MN for the following tasks:

Three multiplication operation
Two addition operation
Eight right-shift operation
One XOR operation
Extraction operation.

So computation overhead of this phase for an MN is:
 $(0.125 \times 3 + 0.125 \times 2 + 0.125 \times 8 + 0.125 + 0.125)\mu s = 1.875 \mu s$.

In the same phase, computation overhead is incurred by a CH for the following tasks:

Extraction operation
One XOR operation
Eight left-shift operation
One Comparison operation.

So computation overhead of this phase for a CH is:
 $(0.125 + 0.125 + 0.125 \times 8 + 0.125)\mu s = 1.375 \mu s$.

Therefore, query response phase (MN–CH) incurs following computational overhead

$$(1.875 + 1.375)\mu s = 3.250 \mu s. \quad (4)$$

Hence the total computation overhead for securing communication between one MN and the CH and between one CH and BS requires:

$$(1) + (2) + (3) + (4) = (1.25 + 0.5 + 3.250 + 3.250)\mu s = 8.25 \mu s.$$

4.3 Communication overhead

As mentioned in Sect. 2.1 we consider that no new nodes are added in the network after the deployment. If the number of member nodes associated with a cluster head is

n and the number of nodes which send query responses is l , then $l \leq n$. Considering energy consumption for transmitting one byte of data is $16.25 \mu J$ and receiving one byte of data is $12.25 \mu J$ for a mica2 mote, the communication overhead of the scheme for query phase and query response phases is evaluated below:

Secured TDMA schedule broadcast

The CH consumes energy for one task- broadcast of TDMA schedule of 5bytes. So total energy consumed for broadcast

$$= (16.25 \times 5) \mu J = 81.25 \mu J \quad (5)$$

The n number of MNs consumes $(12.25 \times 5)n \mu J$
 $= 61.25n \mu J.$ (6)

Query phase (BS–CH)

The CH consumes energy for two tasks- receiving the $T_{BS_present}$ packet and the query packet from BS. Each of the two tasks consumes $12.25 \mu J$ of energy.

Total energy consumed by CH during query phase (BS – CH):
 $(12.25 \times 2) \mu J = 24.5 \mu J.$ (7)

Query phase (CH–MN)

Cluster head consumes energy for four specific tasks – broadcasting of registration packet broadcast time ($T_{CH_present}$), broadcasting of registration packet, receiving registration-response (RR) packets from n number of member nodes (MNs) and transmitting node-id packet (size 1 byte) to the MNs. For the four tasks, it consumes $16.25 \times 2 \mu J$, $16.25 \mu J$, $12.25 \times 4 \times n \mu J$ (RR packet size 4 bytes max) and $16.25 \times n \mu J$ respectively.

Similarly the n numbers of MNs associated with the CH consume energy for four different tasks – receiving the registration packet broadcast time ($T_{CH_present}$) receiving the registration packet, transmitting the RR packet (size 4 bytes approx.) and receiving the node-id. These four tasks consume $12.25 \times 2 \times n \mu J$, $12.25 \times n \mu J$, $16.25 \times 4 \times n \mu J$ and $12.25 \times n \mu J$ respectively.

Therefore, in query phase (CH–MN) total energy consumed by the CH:

$$(16.25 \times 2 + 16.25 + 12.25 \times 4 \times n + 16.25 \times n) \mu J = (48.75 + 65.25n) \mu J \quad (8)$$

and total energy consumed by the MNs:

$$(12.25 \times 2 \times n + 12.25 \times n + 16.25 \times 4 \times n + 12.25 \times n) \mu J = 114n \mu J. \quad (9)$$

Query response phase

Cluster head consumes energy for three tasks – broadcasting of query packet broadcast time (size approx. 2 bytes), broadcasting of query packet (size 1 byte) and receiving query response (QR) packets (size 6 bytes) from l number of member nodes (MNs). For the three tasks, it

Table 1 Comparative performance

Scheme	No. of Keys	Maximum Packet Size (bytes)	Storage overhead (bytes)	Computation overhead (ms)	Communication Overhead (μJ)	Probability of capturing key by adversary during its exchange
SecCBSN	6	36	$40n + 56$ (CH) $32n + 64$ (MNs)	17.12	$585 + 318n + 441l$	Non-zero
SQPS	1	6	$7n$ (CH) $3n$ (MNs)	0.00425	$32.5 + 138.5n + 171l$	Non-zero
SQPS_DT	1	6	$8n + 2$ (CH) $10n$ (MNs)	0.00825	$203.25 + 277.25n + 171l$	Zero

consumes $16.25 \times 2 \mu\text{J}$, $16.25 \mu\text{J}$ and $12.25 \times 6 \times l \mu\text{J}$ respectively.

Further, the n numbers of MNs associated with the CH consume energy for receiving the query packet broadcast time and the query packet whereas l numbers of MNs consume energy for transmitting the QR packet. These three tasks consume $12.25 \times 2 \times n \mu\text{J}$, $12.25 \times n \mu\text{J}$ and $16.25 \times 6 \times l \mu\text{J}$ respectively.

Therefore, in query response phase total energy consumed by the CH:

$$(16.25 \times 2 + 16.25 + 12.25 \times 6 \times l) \mu\text{J} \quad (10)$$

and total energy consumed by the MNs :

$$(12.25 \times 2 \times n + 12.25 \times n + 16.25 \times 6 \times l) \mu\text{J}. \quad (11)$$

So energy consumption of CH considering the three phases (using (5), (7), (8) and (10)):

$$\begin{aligned} &81.25 + 24.5 + (48.75 + 65.25n) \\ &+ (16.25 \times 2 + 16.25 + 12.25 \times 6 \times l) \mu\text{J} \\ &= (81.25 + 122 + 65.25n + 73.5l) \mu\text{J}. \end{aligned} \quad (12)$$

Similarly energy consumption of MNs for both the phases (using (6), (9) and (11)):

$$\begin{aligned} &(61.25n + 114n + 12.25 \times 2 \times n + 12.25 \times n + 16.25 \\ &\times 6 \times l) \mu\text{J} = (61.25n + 150.75n + 97.5l) \mu\text{J}. \end{aligned} \quad (13)$$

Total consumption for CH and MNs (using (12) and (13)):

$$\begin{aligned} &(81.25 + 122 + 65.25n + 73.5l + 61.25n + 150.75n \\ &+ 97.5l) \mu\text{J} = (203.25 + 277.25n + 171l) \mu\text{J} \end{aligned}$$

Table 1 presents the comparative performance of three competing schemes SecCBSN, SQPS and SQPS_DT in terms of the number of keys used, maximum packet size used during transmission, storage overhead, computation overhead, communication overhead and probability of capturing key information by adversary during key exchange. While considering the probability of capturing key information during its exchange, the present scheme SQPS_DT shows its value zero whereas the values are non-zero in the other two schemes. This signifies that the

SQPS_DT has stronger robustness than SQPS and SecCBSN. Although as far as other parameters are concerned, SQPS_DT either requires equal overhead or requires slightly increased overhead compared to SQPS, the table clearly reflects the superiority of SQPS_DT over SecCBSN. SQPS_DT provides stronger robustness compared to SQPS at the cost of this increased overhead.

4.4 Key cracking (SQPS_DT)

As mentioned in Sect. 2.2, we have considered adversaries as mote class attackers which are not capable of launching brute force attacks.

4.4.1 Cracking probability

In query processing phase (Sect. 3), when an MN transmits RR packet using dynamic TDMA, the key does not get transmitted directly. The CH constructs the key after decrypting a part (varying length) of the first part of the RR packet received in first mini-slot. So even if any adversary captures the RR packet, it is not able to get any information about the key i.e. its value, length etc. Therefore, the probability of getting the key is zero for the malicious node. Further, if a node (CH or MN) is compromised, as the adversary has no knowledge about the length of the key it would exhaustively search the stored information in the node to find out the key. In case of such a node compromise attack, the probability of cracking the key is also nearly zero as the key is not getting stored in any node. The computation in the following sub-section as well as the encryption method described in Sect. 3.2 substantiates the claim. As the probability of cracking, whatever small it may be, is non-zero, we argue that the cracking is meaningful only when it starts cracking and finishes it within a same phase (registration/query response). We define this time as meaningful time T_m . After completion of the phase, the cracked key will be of no use. This holds good even if the adversary gets access to the very first node that gets registered by the CH. This is justified and elaborated

further in security analysis (Sect. 4.5). Further, in query processing phase as the key cracking is not possible from transmitted RR packet, the key may be cracked either from the stored data in CH or from MN.

From CH

Considering the size of data stored in CH as S_{DS-CH} , the number of possible combinations:

$$\sum_{j=3}^{S_{DS-CH}-31} 2^j = 2.15 \times 10^9$$

where $S_{DS-CH} = 62$.

The size of data stored at CH is 62 bits. So for average cases, the adversary tries $62/2 = 31$ times to be successful. So j varies up to $S_{DS-CH} - 31$.

The number of computations performed to crack:

$$\sum_{j=3}^{S_{DS-CH}-31} (j-1) = 464.$$

So the key cracking probability from stored data at CH:

$$KCP_{CH} = \frac{1}{\sum_{j=3}^{S_{DS-CH}-31} 2^j} = 4.65 \times 10^{-10}.$$

From MN

Considering the size of data stored in MN as S_{DS-MN} , the number of possible combinations:

$$\sum_{j=3}^{S_{DS-MN}-21} 2^j = 4.19 \times 10^6$$

where $S_{DS-MN} = 42$.

The size of data stored at MN is 42 bits. So for average cases, the adversary tries $42/2 = 21$ times to be successful. So j varies up to $S_{DS-MN} - 21$.

The number of computations performed to crack:

$$\sum_{j=3}^{S_{DS-MN}-21} (j-1) = 230$$

So the key cracking probability from stored data at MN:

$$KCP_{MN} = \frac{1}{\sum_{j=3}^{S_{DS-MN}-21} 2^j} = 2.38 \times 10^{-7}$$

4.4.2 Cracking time

Let the total available bandwidth is BW bits/s, maximum packet size is p bits, and broadcast time by CH is T_B ms. So duration of one TDMA time slot is p/BW s.

From CH

The key computation time (T_c) is computed as follows:

$$T_c = \left[C_t \times \left(\sum_{j=3}^{S_{DS-CH}} (j-1) \right) \times \left(\sum_{j=3}^{S_{DS-CH}} 2^j \right) \right] \\ = \left[C_t \times \left(\sum_{j=3}^{S_{DS-CH}} (j-1) 2^j \right) \right]$$

where C_t is one clock cycle time of mica2 motes.

$$T_c = (0.125 \times 4.2 \times 10^{20}) \mu s$$

$$T_c = 52.5 \times 10^{18} \mu s$$

From MN

The meaningful time (T_m) and key computation time (T_c) are computed as follows:

$$T_m = (n - x) \times p + T_B + (x - 1) \times p/BW$$

where n is the number of member nodes in a cluster and x is the MN which uses x^{th} TDMA time-slot.

$$T_c = \left[C_t \times \left(\sum_{j=3}^{S_{DS-MN}} (j-1) \right) \times \left(\sum_{j=3}^{S_{DS-MN}} 2^j \right) \right] \\ = \left[C_t \times \left(\sum_{j=3}^{S_{DS-MN}} (j-1) 2^j \right) \right]$$

$$T_c = (0.125 \times 230 \times 4194296) \mu s = 12.054 \times 10^7 \mu s.$$

Considering $BW = 1$ Kb/s, $p = 46$ bits, $T_B = 96$ ms, $n = 50$ and $x = 1$.

$$T_m = (49 \times 46) \text{ ms} + 96 \text{ ms} = 2,254 \text{ ms} + 96 \text{ ms} \\ = 2,350 \text{ ms} = 23,50,000 \mu s.$$

The above computations illustrate that irrespective of type of the nodes (CH or MN), if the nodes are compromised, time required to compute the key for malicious node is always greater than the meaningful time and hence the key will be of no use. Therefore, we may consider the probability of capturing the key in presence of node compromise attack is also zero.

4.5 Security analysis

The prime objective of the present scheme is to provide robust security to the network. In both SQPS and SQPS_DT, after the completion of every round a new cluster head is elected based on the LEACH protocol. In schedule broadcast phase, the concept of pre-deployed key is used. As mentioned in Sect. 2, this key is a single network wide key [17] that is pre-loaded in all the nodes of the network prior to deployment. If the key is captured, an adversary can successfully use it only in schedule broadcast phase (Sect. 2.3) as in no other phase this key is used. It is already shown in 2.3 that this phase can thwart replay attack.

In data transfer/query processing phase, the key used for communication between a cluster head and member nodes are generated by the member nodes during registering the member nodes by the cluster head in Query (CH–MN) phase. Subsequent to key generation, in SQPS concept of fixed TDMA has been used for member nodes to send the keys as part of the respective registration-response (RR) packets to their cluster heads. So if a malicious node can get hold of a RR packet in spite of the fact that key is intermingled with other data, probability of extracting it by the malicious node is non-zero. On the other hand, in SQPS_DT, member nodes send RR packets containing keys in encrypted form of varying length using dynamic TDMA. The cluster head gains knowledge of individual key of every member node after performing the required decryption on a part (varying length) of the first part of RR packet sent in first mini-slot by individual member nodes as described in Sect. 3.2. In a word, the key is neither present in the nodes at the time of deployment nor it is communicated through packet exchange between cluster head and member nodes. Although during registering of MNs, S is transmitted directly through medium and S' is intermingled with other data of the RR packet and the packet is transmitted in two parts, the probability of obtaining S' is $1/2^{20} = 0.00000095$ (non-zero). So a malicious node, even after some observations on S and S' cannot guess the key (3-bit) as it is sent not only in encrypted form but also encrypted key is of varying length. In spite of this, if the adversary obtains the key, the information will be of no use as in the next Query Response phase though same key is used but a different encryption procedure (right shifting) is used. Moreover, in the next query phase the member nodes generate key afresh. Also, additional security in the form of base station authentication is provided in SQPS_DT as mentioned in Sect. 3.1. This prevents malicious nodes to pose as base station and send bogus queries or flood the network with fake queries. The cluster heads accept query only from authenticated base station. These unique features of SQPS_DT enables to provide for enhanced robustness to the security aspect of the network and no adversary can obtain the key even with the use of complex cryptographic techniques. Also both in SQPS and SQPS_DT, new key set up in the network are considered after every round reducing the probability of gaining knowledge about the keys.

In SecCBSN, the primary security module is involved with the security of nodes while they are being introduced into the network. Before deployment, every node is loaded with an individual key chain ($< tK >$), a private function (f_s), an individual key (K_s) and a certificate ($TCert_s$). All of these parameters are required by a new node joining the network for establishing its authenticity. Existing nodes send their key derivations ($f(ID_s)$) to neighboring new nodes for establishing pairwise keys with them. These

pairwise keys are derived by the new nodes using the key derivation received from the already existing neighbouring nodes and their private function (f_s). So a lot of information has to be incorporated into a node in pre-deployment stage. Moreover, many complicated operations with the help of pre-deployed information need to be performed by any node for deriving the other keys used in the network. So these operations lead to the consumption of a lot of energy which is a scarce resource in WSN.

Moreover, both in SQPS and SQPS_DT even if a member node gets compromised by an adversary, the secret information that is revealed to the adversary is exclusively for that node only. As the keys are not shared there is no chance that the keys of other nodes are exposed. So even though the communication between the compromised node and cluster head is hampered, the communication between the non-compromised nodes and cluster head is not affected. Therefore, no adversary can obtain any security related information with the help of which it can disturb the operation of the entire network. However, in SecCBSN, if any malicious node is able to compromise authentic nodes already present in the network, it can gain knowledge of all the information stored in the compromised node and use these information to establish pairwise keys with new incoming nodes. Thus new nodes are duped into believing the malicious node as an authentic one and continue communication with the adversary node. So this a major flaw of SecCBSN as authenticity of new nodes are established by their neighboring nodes which themselves can be malicious nodes. Therefore, a malicious node posing as an authentic one by compromising a genuine node, can allow the intrusion of new malicious nodes into the network, if these new malicious nodes are placed in the network as its neighbours. In this manner groups of malicious nodes can form several sub-networks within the network and propagate false information to the cluster head nodes. So neither the cluster head node nor the monitor node is aware of this, as transmission of false information cannot be detected by either the cluster head or the monitor node.

5 Conclusion

In this paper we have proposed a secured query processing scheme to ensure the basic security properties such as authentication, data integrity and data freshness as well as protect the WSN from replay attacks. The proposed security solution is lightweight as it uses symmetric key cryptography using very simple keys.

The query processing is robust from security point of view as the key used for securing communication among different nodes of the network is neither pre-deployed nor

is transmitted directly. Moreover, the key is generated randomly at each member node and is encrypted to generate an encrypted key of varying length. Further, the encrypted key is transmitted along with other data using one variant of dynamic TDMA, which makes the key virtually impossible to decipher. Another variant of dynamic TDMA is employed for bandwidth utilization.

Performance analysis of the scheme reveals its suitability for sensor networks as relatively less overhead is incurred in respect of storage, computation and communication requirements of nodes. The overhead incurred for this scheme computed through performance analysis which reveals its suitability for sensor networks as relatively less overhead is incurred in terms of storage, computation and communication requirements of nodes. Moreover the scheme is compared with its previous version [29] and one of the existing security schemes based on the results of performance analysis, which justify its efficacy in respect to all the comparison parameters.

As a future extension, the scheme may be made more general by making it logical-topology free. Moreover, the solution may be made more realistic allowing new nodes to join the network at any point of time.

References

1. Akyildiz, I. F., Su, W., Sankarabramanian, Y., & Cayirci, E. (2002). A survey on sensor networks. *IEEE Communications Magazine*, 40(8), 102–114.
2. DasBit, S., & Ragupathy, R. (2008). Routing in MANET and sensor network—A 3D position based approach. *Journal of Foundation of Computing and Decision Sciences*, 33(3), 211–239.
3. Ghosal, A., Halder, S., & DasBit, S. (2009). A scheme to tolerate jamming in multiple nodes in wireless sensor networks. *Proceedings of Wireless VITAE*, IEEE press, 948–951.
4. Ghosal, A., Halder, S., Chatterjee, S., Sen, J., & DasBit, S. (2009). Estimating delay in a data forwarding scheme for defending jamming attack in wireless sensor network. *Proceedings of 3rd International Conference NGMAST*, IEEE CS press, 351–356.
5. Halder, S., Ghosal, A., Sur, S., Dan, A., & DasBit, S. (2009). A lifetime enhancing node deployment strategy in WSN. *Proceedings of Future Generation Information Technology*, LNCS-5899, 296–308.
6. Heinzelman, W. R., Chandrakasan, A., & Balakrishnan, H. (2000). Energy efficient communication protocol for wireless microsensor networks. *Proceedings of 33rd Hawaii International Conference on System Sciences*, 2, 8020–8029.
7. Ferreira, A. C., Vilaca, M. A., Oliveira, L. B., Habib, E., Wang, H. C., & Loureiro, A. A. (2005). On the security of cluster-based communication protocols for wireless sensor networks. *Proceedings of 4th International Conference on Networking*, LNCS-3420, 449–458.
8. Wood, A. D., & Stankovic, J. A. (2002). Denial of service in sensor networks. *IEEE Computer*, 35(10), 54–62.
9. Crosby, G. V., Pissinou, N., & Gadze, J. (2006). A framework for trust-based cluster head election in wireless sensor networks. *Proceedings of 2nd IEEE Workshop on Dependability and Security in Sensor Networks and Systems*, 13–22.
10. Lee, P., Kim, J., Han, I., Rye, H., & Ahn, W. H. (2007). Secure cluster header election techniques in sensor network. *Proceedings of ISPA Workshop*, LNCS-4743, 20–31.
11. Oliveira, L. B., Wong, H. C., Bern, M., Dahab, R., & Loureiro, A. A. F. (2006). SecLEACH—A random key distribution solution for securing clustered sensor networks. *Proceedings of 5th IEEE International Symposium on Network Computing and Applications*, 145–154.
12. Hsieh, M. Y., Huang, Y. M., & Chao, H. C. (2007). Adaptive security design with malicious node detection in cluster-based sensor networks. *Journal of Computer Communications*, 30(11–12), 2385–2400.
13. Poornima, A. S., & Amberker, B. B. (2009). Protocols for secure node-to-cluster head communication in clustered wireless sensor networks. *Proceedings of 2nd International Conference on Contemporary Computing*, CCIS-40, 434–444.
14. Kao, J. C., & Marculescu, R. (2007). Minimizing eavesdropping risk by transmission power control in multihop wireless networks. *IEEE Transaction on Computers*, 56(8), 1009–1023.
15. Kao, J. C., & Marculescu, R. (2007). Energy-efficient anonymous multicast in mobile ad-hoc networks. *Proceedings of 13th International Conference on Parallel and Distributed Systems*, 1, 1–8.
16. Bashar, S., & Ding, Z. (2009). Optimum power allocation against information leakage in wireless network. *Proceedings of 28th IEEE GLOBECOM*, 1–6.
17. Xiao, Y., Rayi, V. K., Sun, B., Xiaojian, D., Fei, H., & Galloy, M. (2007). A survey of key management schemes in wireless sensor networks. *Journal of Computer Communications*, 30(11–12), 2314–2341.
18. Jiang, C.-j., Shi, W.-r., Xiang, M., & Tang, X.-L. (2010). Energy-balanced unequal clustering protocol for wireless sensor networks. *The Journal of China Universities of Posts and Telecommunications*, 17(4), 94–99.
19. Deng, J., Han, R., & Mishra, S. (2006). Limiting DoS attacks during multihop data delivery in wireless sensor networks. *Journal of Security and Networks*, 1(3/4), 167–178.
20. David, R. R., Marchany, R. C., & Midkiff, S. F. (2007). Scalable, Cluster-based Anti-replay Protection for Wireless Sensor Networks. *Proceedings of the 2007 IEEE Workshop on Information Assurance*, 127–134.
21. Murthy, C. S. R., & Manoj, B. S. (2004). *Ad hoc wireless networks architectures and protocols*. New Jersey: Prentice Hall.
22. Time Division Multiple Access. http://en.wikipedia.org/wiki/Time_division_multiple_access. Accessed 15 March 2010.
23. Kim, B. C., & Un, C. K. (1994). Capacity of wireless dynamic TDMA and media access control for packetized voice/data integration. *Journal of Wireless Personal Communications*, 1(4), 313–319.
24. Drakopoulos, E., & Gerakoulis, D. (1998). Performance of a dynamic TDMA system for mobile wireless access. *International Journal of Wireless Information Networks*, 5(4), 321–340.
25. Wu, C. M. (2007). Dynamic frame length channel assignment in wireless multihop ad hoc networks. *Journal of Computer Communications*, 30(18), 3832–3840.
26. Pomalaza-Raez, C. A., & Hemminger, T. L. (2002). A unified approach to dynamic TDMA slot assignment and to distributed routing for multi-hop packet radio networks. *Proceedings of ANNIE Conference*, 975–980.
27. Ren, F. C., Chang, C. J., & Cheng, R. G. (2002). QoS-guaranteed fuzzy transmission controller for dynamic TDMA protocol in

- multimedia communication. *IEEE Proceedings-Communications*, 149(6), 292–298.
28. Zia, T., & Zomaya, A. (2006). Security issues in wireless sensor networks. *Proceedings of International Conference on Systems and Network Communications*, 1–4.
 29. Atmel Corporation Product Document (2009). http://www.atmel.com/dyn/resources/prod_documents/doc2467.pdf. Accessed 15 March 2010.
 30. Ghosal, A., Halder, S., Sur, S., Dan, A., & DasBit, S. (2010). Ensuring basic security and preventing replay attack in a query processing application domain in WSN. *Proceedings of International Conference on Computational Science and its Application (ICCSA), LNCS-6018*, 321–335.

Author Biographies



Amrita Ghosal is an Assistant Professor in the Department of Computer Science and Engineering, Dr. B.C. Roy Engineering College, Durgapur, India. She received B.Tech. in Electronics and Communication Engineering from Asansol Engineering College, and M.Tech. in Computer Science and Engineering from Kalyani Government Engineering College, Kalyani, India in 2003 and 2006 respectively. She is currently pursuing Ph.D. at the

Department of Computer Science and Technology, Bengal Engineering and Science University, Shibpur, West Bengal, India. Her current area of research is wireless sensor networks. Her main areas of interest include detection, prevention and cure of different DoS style attacks in wireless sensor networks.



Subir Halder is an Assistant Professor in the Department of Computer Science and Engineering, Dr. B.C. Roy Engineering College, Durgapur, India. He received B. Tech. in Electronics and Communication Engineering and M.Tech. in Computer Science and Engineering from Kalyani Government Engineering College, Kalyani, India in 2003 and 2006 respectively. He is currently pursuing Ph.D. at the Department of Computer Science and

Technology, Bengal Engineering and Science University, Shibpur, West Bengal, India. His areas of interest include issues related to coverage, connectivity, QoS, energy efficient node deployment in wireless sensor networks.



Sipra DasBit is a Professor in the Department of Computer Science and Technology, Bengal Engineering and Science University, Shibpur, West Bengal, India. A recipient of the Career Award for Young Teachers from the All India Council of Technical Education (AICTE), she has more than 20 years of teaching and research experience. Professor DasBit has published many research papers in reputed journals and refereed International

conference proceedings. She also has one book and book chapter on mobile computing to her credit. Her current research interests include wireless sensor networks and mobile computing.